RAG with HuggingFace and Milvus - Complete Implementation

Domain-Specific RAG Pipeline for HuggingFace Documentation

Dataset: `m-ric/huggingface_doc`
Vector Store: Milvus Lite (local file-based)
Embeddings: BAAI/bge-small-en-v1.5 (384-dim, normalized)
LLM: Qwen/Qwen2-1.5B-Instruct
Evaluation: Opik (AnswerRelevance, Hallucination)

---

1. Setup

Install required dependencies and configure environment.

```
# Install dependencies
!pip install -q pymilvus sentence-transformers datasets transformers torch accelerate opik tqdm
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 152.9/152.9 kB 2.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 301.2/301.2 kB 10.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 25.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 69.8/69.8 kB 3.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.9/14.9 MB 51.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 28.1 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 278.1/278.1 kB 5.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 66.8/66.8 kB 1.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.3/43.3 kB 735.9 kB/s eta 0:00:00
```

```
import os
import json
from typing import List, Dict, Tuple
from tqdm import tqdm

# Set your HuggingFace token for model access
# Get one at: https://huggingface.co/settings/tokens
os.environ["HF_TOKEN"] = "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"  # <-- Replace with your HF token

# Opik configuration for evaluation
# Get your API key at: https://www.comet.com/
os.environ["OPIK_API_KEY"] = "sk-proj-tTLsVbJVj-els-cQOQ1qMDnCOSRiRslZ7qnuIuc3grHIzjD6BFclR_woqcK9xyqB7VBsz0QYulT3BlbkFJYWk

print("Environment configured!")
```

```
Environment configured!
```

2. Data Loading

Load the HuggingFace documentation dataset from `m-ric/huggingface_doc`.

```
from datasets import load_dataset

# Load the HuggingFace documentation dataset
dataset = load_dataset("m-ric/huggingface_doc", split="train")

print(f"Dataset loaded with {len(dataset)} documents")
print(f"Columns: {dataset.column_names}")
print(f"\nSample document (first 500 chars):")
print(dataset[0]["text"][:500])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

```
README.md: 100%                                    21.0/21.0 [00:00<00:00, 377B/s]

huggingface_doc.csv: 100%                          22.0M/22.0M [00:01<00:00, 189MB/s]

Generating train split: 100%                       2647/2647 [00:00<00:00, 3045.91 examples/s]
```

```
Dataset loaded with 2647 documents
Columns: ['text', 'source']

Sample document (first 500 chars):
 Create an Endpoint

After your first login, you will be directed to the [Endpoint creation page](https://ui.endpoints.huggingface.co/new). As ar

## 1. Enter the Hugging Face Repository ID and your desired endpoint name:

<img src="https://raw.githubusercontent.com/huggingface/hf-endpoints-docu
```

```
# Extract text and source information
documents = []
for item in dataset:
    documents.append({
        "text": item["text"],
        "source": item["source"]
    })

print(f"Extracted {len(documents)} documents")

# Use a subset to keep things manageable
MAX_DOCS = 500
documents = documents[:MAX_DOCS]
print(f"Using {len(documents)} documents for this assignment")
```

```
Extracted 2647 documents
Using 500 documents for this assignment
```

3. Chunking

Split documents into overlapping chunks using a **sliding window** approach.

Design choices:

- `chunk_size=1000`: Large enough to capture complete concepts, small enough for precise retrieval
- `chunk_overlap=200`: 20% overlap preserves semantic continuity at chunk boundaries
- Metadata (`source`, `chunk_id`) is propagated to every chunk for lineage tracking

```
def chunk_document(text: str, chunk_size: int = 1000, chunk_overlap: int = 200) -> List[str]:
    """
    Split a document into overlapping chunks using a sliding window.

    Args:
        text: The document text to chunk
        chunk_size: Maximum size of each chunk in characters
        chunk_overlap: Number of overlapping characters between consecutive chunks

    Returns:
        List of text chunk strings
    """
    chunks = []

    # Step 1: Handle edge cases
    if not text or not text.strip():
        return chunks

    if len(text) <= chunk_size:
        chunks.append(text.strip())
        return chunks

    # Step 2: Calculate sliding window step size
    step = chunk_size - chunk_overlap

    # Step 3: Slide through the text
```

```
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]

        # Step 4: Only add non-empty chunks
        if chunk.strip():
            chunks.append(chunk.strip())

        start += step

    return chunks


def chunk_all_documents(documents: List[Dict], chunk_size: int = 1000, chunk_overlap: int = 200) -> List[Dict]:
    """
    Chunk all documents and preserve source metadata.

    Args:
        documents: List of dicts with 'text' and 'source' keys
        chunk_size: Maximum chunk size in characters
        chunk_overlap: Overlap between consecutive chunks

    Returns:
        List of chunk dicts with 'chunk_id', 'text', and 'source'
    """
    all_chunks = []
    chunk_id = 0

    for doc in tqdm(documents, desc="Chunking documents"):
        text = doc["text"]
        source = doc["source"]

        doc_chunks = chunk_document(text, chunk_size, chunk_overlap)

        for chunk_text in doc_chunks:
            all_chunks.append({
                "chunk_id": chunk_id,
                "text": chunk_text,
                "source": source
            })
            chunk_id += 1

    return all_chunks
```

```
# Test chunking implementation
test_text = "A" * 2500  # 2500 characters
test_chunks = chunk_document(test_text, chunk_size=1000, chunk_overlap=200)

print(f"Test: 2500 char text with chunk_size=1000, overlap=200")
print(f"Expected chunks: ~4")
print(f"Your chunks: {len(test_chunks)}")

if 3 <= len(test_chunks) <= 5:
    print(" Chunking test passed!")
else:
    print(" Check your chunking implementation")
```

```
Test: 2500 char text with chunk_size=1000, overlap=200
Expected chunks: ~4
Your chunks: 4
 Chunking test passed!
```

```
# Create chunks from all documents
CHUNK_SIZE = 1000
CHUNK_OVERLAP = 200

chunks = chunk_all_documents(documents, CHUNK_SIZE, CHUNK_OVERLAP)

print(f"\nCreated {len(chunks)} chunks from {len(documents)} documents")
print(f"Average chunks per document: {len(chunks) / len(documents):.2f}")

if chunks:
    print(f"\nSample chunk:")
    print(f"  ID: {chunks[0]['chunk_id']}")
    print(f"  Source: {chunks[0]['source']}")
    print(f"  Text (first 200 chars): {chunks[0]['text'][:200]}...")
```

```
Chunking documents: 100%|██████████| 500/500 [00:00<00:00, 24434.64it/s]
Created 5651 chunks from 500 documents
Average chunks per document: 11.30
```

```
Sample chunk:
  ID: 0
  Source: huggingface/hf-endpoints-documentation/blob/main/docs/source/guides/create_endpoint.mdx
  Text (first 200 chars): Create an Endpoint

After your first login, you will be directed to the [Endpoint creation page](https://ui.endpoints.huggingface.co/new). As ar
```

4. Embeddings

Generate vector embeddings using **BAAI/bge-small-en-v1.5** (384 dimensions).

Design choices:

- `normalize_embeddings=True`: L2-normalizes vectors so Inner Product (IP) equals cosine similarity
- `batch_size=32`: Balances GPU memory usage vs throughput
- BGE-small is optimized for retrieval tasks and produces high-quality embeddings at low compute cost

```
from sentence_transformers import SentenceTransformer

# Load the embedding model
EMBEDDING_MODEL = "BAAI/bge-small-en-v1.5"
embedding_model = SentenceTransformer(EMBEDDING_MODEL)

print(f"Loaded embedding model: {EMBEDDING_MODEL}")

# Determine embedding dimension
test_embedding = embedding_model.encode(["This is a test"], normalize_embeddings=True)
EMBEDDING_DIM = len(test_embedding[0])
print(f"Embedding dimension: {EMBEDDING_DIM}")
```

```
modules.json: 100%  349/349 [00:00<00:00, 22.6kB/s]
config_sentence_transformers.json: 100%  124/124 [00:00<00:00, 4.38kB/s]
README.md:  94.8k/? [00:00<00:00, 2.09MB/s]
sentence_bert_config.json: 100%  52.0/52.0 [00:00<00:00, 1.13kB/s]
config.json: 100%  743/743 [00:00<00:00, 14.7kB/s]
model.safetensors: 100%  133M/133M [00:01<00:00, 99.7MB/s]
Loading weights: 100%  199/199 [00:00<00:00, 306.85it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: BAAI/bge-small-en-v1.5
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%  366/366 [00:00<00:00, 12.3kB/s]
vocab.txt:  232k/? [00:00<00:00, 3.56MB/s]
tokenizer.json:  711k/? [00:00<00:00, 9.26MB/s]
special_tokens_map.json: 100%  125/125 [00:00<00:00, 2.83kB/s]
config.json: 100%  190/190 [00:00<00:00, 2.43kB/s]
Loaded embedding model: BAAI/bge-small-en-v1.5
Embedding dimension: 384
```

```
def generate_embeddings(texts: List[str], model: SentenceTransformer, batch_size: int = 32) -> List[List[float]]:
    """
    Generate normalized embeddings for a list of texts in batches.

    Args:
        texts: List of text strings to embed
        model: SentenceTransformer model instance
        batch_size: Number of texts to process per batch

    Returns:
        List of embedding vectors (as Python lists of floats)
    """
    all_embeddings = []

    # Process in batches for memory efficiency
    for i in tqdm(range(0, len(texts), batch_size), desc="Generating embeddings"):
        batch = texts[i : i + batch_size]
```

```
        # Encode with L2 normalization so IP == cosine similarity
        batch_embeddings = model.encode(
            batch,
            normalize_embeddings=True,
            show_progress_bar=False
        )

        # Convert numpy array to Python list for Milvus compatibility
        all_embeddings.extend(batch_embeddings.tolist())

    return all_embeddings
```

```
# Test embedding generation
test_texts = ["Hello world", "This is a test", "RAG is cool"]
test_embeddings = generate_embeddings(test_texts, embedding_model)

print(f"Generated {len(test_embeddings)} embeddings")
print(f"Embedding dimension: {len(test_embeddings[0]) if test_embeddings else 0}")

if len(test_embeddings) == 3 and len(test_embeddings[0]) == 384:
    print(" Embedding generation test passed!")
else:
    print(" Check your embedding implementation")
```

```
Generating embeddings: 100%|██████████| 1/1 [00:00<00:00, 11.44it/s]Generated 3 embeddings
Embedding dimension: 384
 Embedding generation test passed!
```

```
# Generate embeddings for all chunks
chunk_texts = [chunk["text"] for chunk in chunks]
embeddings = generate_embeddings(chunk_texts, embedding_model)

print(f"\nGenerated {len(embeddings)} embeddings")
if embeddings:
    print(f"Embedding dimension: {len(embeddings[0])}")
    print(f"Sample embedding (first 10 values): {embeddings[0][:10]}")
```

```
Generating embeddings: 100%|██████████| 177/177 [46:50<00:00, 15.88s/it]
Generated 5651 embeddings
Embedding dimension: 384
Sample embedding (first 10 values): [-0.07532963901758194, -0.027507977560162544, -0.039956092834472656, -0.040492128580808
```

5. Vector Store (Milvus)

Store embeddings in **Milvus Lite** for efficient similarity search.

Design choices:

- `metric_type="IP"` (Inner Product): Since embeddings are L2-normalized, IP is equivalent to cosine similarity
- `consistency_level="Strong"`: Ensures reads always reflect the latest writes (important for correctness)
- Milvus Lite stores everything in a local `.db` file — no server required
- Batched insertion (`batch_size=100`) avoids memory spikes during large ingestion

Milvus Schema:

Field	Type	Description
`id`	INT64 (PK)	Unique chunk identifier
`vector`	FLOAT_VECTOR(384)	BGE embedding
`text`	VARCHAR	Chunk text content
`source`	VARCHAR	Source document URL/path

```
!pip install pymilvus[milvus_lite]
```

```
Requirement already satisfied: pymilvus[milvus_lite] in /usr/local/lib/python3.12/dist-packages (2.6.9)
Requirement already satisfied: setuptools>69 in /usr/local/lib/python3.12/dist-packages (from pymilvus[milvus_lite]) (75.2.0
Requirement already satisfied: grpcio!=1.68.0,!=1.68.1,!=1.69.0,!=1.70.0,!=1.70.1,!=1.71.0,!=1.72.1,!=1.73.0,>=1.66.2 in /us
Requirement already satisfied: orjson>=3.10.15 in /usr/local/lib/python3.12/dist-packages (from pymilvus[milvus_lite]) (3.11
Requirement already satisfied: protobuf>=5.27.2 in /usr/local/lib/python3.12/dist-packages (from pymilvus[milvus_lite]) (5.2
Requirement already satisfied: python-dotenv<2.0.0,>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from pymilvus[milvus_
Requirement already satisfied: pandas>=1.2.4 in /usr/local/lib/python3.12/dist-packages (from pymilvus[milvus_lite]) (2.2.2)
Requirement already satisfied: cachetools>=5.0.0 in /usr/local/lib/python3.12/dist-packages (from pymilvus[milvus_lite]) (6.
Collecting milvus-lite>=2.4.0 (from pymilvus[milvus_lite])
  Downloading milvus_lite-2.5.1-py3-none-manylinux2014_x86_64.whl.metadata (10.0 kB)
Requirement already satisfied: typing-extensions~=4.12 in /usr/local/lib/python3.12/dist-packages (from grpcio!=1.68.0,!=1.6
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from milvus-lite>=2.4.0->pymilvus[milvus_lit
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2.4->pymilvus[milvus
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2.4->pymilv
```

```
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2.4->pymilvus[milvus_
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2.4->pymilvus[milvu
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.2
Downloading milvus_lite-2.5.1-py3-none-manylinux2014_x86_64.whl (55.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 55.3/55.3 MB 10.5 MB/s eta 0:00:00
Installing collected packages: milvus-lite
Successfully installed milvus-lite-2.5.1
```

```
from pymilvus import MilvusClient

# Initialize Milvus Lite client — stores data in a local file
MILVUS_DB_PATH = "./hf_docs_milvus.db"
milvus_client = MilvusClient(uri=MILVUS_DB_PATH)

COLLECTION_NAME = "hf_documentation"

print(f"Milvus client initialized with database: {MILVUS_DB_PATH}")
```

```
Milvus client initialized with database: ./hf_docs_milvus.db
```

```
def setup_milvus_collection(client: MilvusClient, collection_name: str, embedding_dim: int):
    """
    Create a Milvus collection for storing document embeddings.

    Args:
        client: MilvusClient instance
        collection_name: Name of the collection to create
        embedding_dim: Dimension of the embedding vectors
    """
    # Step 1: Drop existing collection if it exists (fresh start)
    if client.has_collection(collection_name):
        client.drop_collection(collection_name)
        print(f"Dropped existing collection: {collection_name}")

    # Step 2: Create new collection
    # metric_type="IP" works as cosine similarity for normalized vectors
    client.create_collection(
        collection_name=collection_name,
        dimension=embedding_dim,
        metric_type="IP",
        consistency_level="Strong"
    )

    print(f"Created collection: {collection_name} with dimension {embedding_dim}")
```

```
# Setup the Milvus collection
setup_milvus_collection(milvus_client, COLLECTION_NAME, EMBEDDING_DIM)
```

```
Created collection: hf_documentation with dimension 384
```

```
def insert_data_to_milvus(
    client: MilvusClient,
    collection_name: str,
    chunks: List[Dict],
    embeddings: List[List[float]],
    batch_size: int = 100
) -> int:
    """
    Insert document chunks and their embeddings into Milvus.

    Args:
        client: MilvusClient instance
        collection_name: Name of the target collection
        chunks: List of chunk dicts with 'chunk_id', 'text', 'source'
        embeddings: Corresponding embedding vectors
        batch_size: Number of records to insert per batch

    Returns:
        Total number of records inserted
    """
    total_inserted = 0

    # Step 1: Prepare records as list of dicts
    data = [
        {
            "id": chunk["chunk_id"],
            "vector": embedding,
            "text": chunk["text"],
            "source": chunk["source"]
        }
```

```
        for chunk, embedding in zip(chunks, embeddings)
    ]

    # Step 2: Insert in batches
    for i in tqdm(range(0, len(data), batch_size), desc="Inserting into Milvus"):
        batch = data[i : i + batch_size]
        result = client.insert(collection_name=collection_name, data=batch)
        total_inserted += result["insert_count"]

    return total_inserted
```

```
# Insert all data into Milvus
inserted_count = insert_data_to_milvus(milvus_client, COLLECTION_NAME, chunks, embeddings)

print(f"\nInserted {inserted_count} records into Milvus")

if inserted_count == len(chunks):
    print(" All chunks inserted successfully!")
else:
    print(" Not all chunks were inserted. Check your implementation.")
```

```
Inserting into Milvus: 100%|██████████| 57/57 [00:01<00:00, 29.74it/s]
Inserted 5651 records into Milvus
 All chunks inserted successfully!
```

6. Retrieval

Semantic search over the Milvus vector store.

How it works:

Query text → embed with the same BGE model
Normalized query vector → Inner Product search in Milvus
Return top-K most similar chunks with their text, source, and similarity score

```
def retrieve_documents(
    query: str,
    client: MilvusClient,
    collection_name: str,
    embedding_model: SentenceTransformer,
    top_k: int = 5
) -> List[Dict]:
    """
    Retrieve the most relevant document chunks for a query.

    Args:
        query: Natural language search query
        client: MilvusClient instance
        collection_name: Name of the collection to search
        embedding_model: Model to generate query embedding
        top_k: Number of results to return

    Returns:
        List of dicts with 'text', 'source', and 'score' keys
    """
    # Step 1: Embed the query (must use same model and normalization as ingestion)
    query_embedding = embedding_model.encode(
        [query],
        normalize_embeddings=True
    ).tolist()[0]

    # Step 2: Search Milvus for nearest neighbors via Inner Product
    results = client.search(
        collection_name=collection_name,
        data=[query_embedding],
        limit=top_k,
        search_params={"metric_type": "IP", "params": {}},
        output_fields=["text", "source"]
    )

    # Step 3: Format results as list of dicts
    retrieved_docs = []
    for result in results[0]:  # results[0] because we searched with one query
        retrieved_docs.append({
            "text": result["entity"]["text"],
            "source": result["entity"]["source"],
            "score": result["distance"]
        })
```

```
    return retrieved_docs
```

```
# Test retrieval
test_query = "How do I fine-tune a transformer model?"

retrieved = retrieve_documents(
    query=test_query,
    client=milvus_client,
    collection_name=COLLECTION_NAME,
    embedding_model=embedding_model,
    top_k=3
)

print(f"Query: {test_query}")
print(f"\nRetrieved {len(retrieved)} documents:")
for i, doc in enumerate(retrieved):
    print(f"\n--- Document {i+1} (Score: {doc.get('score', 'N/A'):.4f}) ---")
    print(f"Source: {doc.get('source', 'N/A')}")
    print(f"Text: {doc.get('text', 'N/A')[:300]}...")

if len(retrieved) == 3 and all('text' in d for d in retrieved):
    print("\n✅ Retrieval test passed!")
else:
    print("\n❌ Check your retrieval implementation")
```

```
Query: How do I fine-tune a transformer model?

Retrieved 3 documents:

--- Document 1 (Score: 0.8237) ---
Source: huggingface/blog/blob/main/vision_language_pretraining.md
Text: models from Transformers.*...

--- Document 2 (Score: 0.7484) ---
Source: huggingface/blog/blob/main/ray-rag.md
Text: ects/rag/finetune_rag_ray.sh) for faster distributed fine-tuning, you can leverage RAG for retrieval-based generation


Also, hyperparameter tuning is another aspect of transformer fine tuning and can have [huge impacts on accuracy](https://mec

--- Document 3 (Score: 0.7303) ---
Source: huggingface/blog/blob/main/lewis-tunstall-interview.md
Text: n try to integrate it into your application.

So what I've been working on for the last few months on the transformers library is providing the functionality to export th

✅ Retrieval test passed!
```

7. Generation

Generate answers using **Qwen/Qwen2-1.5B-Instruct** grounded in retrieved context.

Hallucination Control:

The prompt explicitly restricts the model to only use the provided `<context>` tags
If the context is insufficient, the model is instructed to say so instead of hallucinating
`temperature=0.7`, `top_p=0.9`: Balanced sampling for fluency without over-creativity

```
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
import torch

# Qwen2-1.5B-Instruct: small, fast, instruction-tuned
# Trade-off: lighter weight than Phi-3-mini, faster inference, slightly lower quality
LLM_MODEL = "Qwen/Qwen2-1.5B-Instruct"

print(f"Loading model: {LLM_MODEL}")
print("This may take a few minutes...")

tokenizer = AutoTokenizer.from_pretrained(LLM_MODEL, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    LLM_MODEL,
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="auto",
    trust_remote_code=True
)

# Create text generation pipeline
generator = pipeline(
    "text-generation",
```

```
    model=model,
    tokenizer=tokenizer
)

print(f"Model loaded successfully!")
```

```
Loading model: Qwen/Qwen2-1.5B-Instruct
This may take a few minutes...
```

config.json: 100% 660/660 [00:00<00:00, 46.7kB/s]

tokenizer_config.json: 1.29k/? [00:00<00:00, 78.6kB/s]

vocab.json: 2.78M/? [00:00<00:00, 28.6MB/s]

merges.txt: 1.67M/? [00:00<00:00, 32.3MB/s]

tokenizer.json: 7.03M/? [00:00<00:00, 65.1MB/s]

```
`torch_dtype` is deprecated! Use `dtype` instead!
```

model.safetensors: 100% 3.09G/3.09G [00:35<00:00, 206MB/s]

Loading weights: 100% 338/338 [00:22<00:00, 18.18it/s, Materializing param=model.norm.weight]

generation_config.json: 100% 242/242 [00:00<00:00, 19.6kB/s]

```
Model loaded successfully!
```

```
# Prompt template with explicit grounding and hallucination guardrail
PROMPT_TEMPLATE = """Use the following pieces of information enclosed in <context> tags to provide an answer to the questic
If the context doesn't contain enough information to answer the question, say "I don't have enough information to answer th
Do NOT make up information that is not in the context.

<context>
{context}
</context>

<question>
{question}
</question>

Answer:"""
```

```
def generate_answer(
    query: str,
    retrieved_docs: List[Dict],
    generator,
    max_new_tokens: int = 256
) -> Dict:
    """
    Generate a grounded answer using retrieved documents as context.

    Args:
        query: The user's question
        retrieved_docs: List of retrieved document dicts
        generator: HuggingFace text-generation pipeline
        max_new_tokens: Maximum tokens to generate

    Returns:
        Dict with 'answer', 'context', 'query', and 'retrieved_docs'
    """
    # Step 1: Combine retrieved documents into a single context string
    context = "\n\n".join([doc["text"] for doc in retrieved_docs])

    # Step 2: Format the prompt using the grounding template
    prompt = PROMPT_TEMPLATE.format(context=context, question=query)

    # Step 3: Generate response
    outputs = generator(
        prompt,
        max_new_tokens=max_new_tokens,
        do_sample=True,
        temperature=0.7,
        top_p=0.9,
        return_full_text=False
    )

    # Step 4: Extract the generated text
    answer = outputs[0]["generated_text"].strip()

    # Step 5: Return structured result
    return {
        "query": query,
        "answer": answer,
```

```
        "context": context,
        "retrieved_docs": retrieved_docs
    }
```

```
def generate_answer(
    query: str,
    retrieved_docs: List[Dict],
    generator,
    max_new_tokens: int = 128
) -> Dict:

    # Truncate chunks to keep prompt short and fast
    context = "\n\n".join([doc["text"][:300] for doc in retrieved_docs])

    prompt = PROMPT_TEMPLATE.format(context=context, question=query)

    outputs = generator(
        prompt,
        max_new_tokens=max_new_tokens,
        do_sample=False,                        # greedy = fast
        return_full_text=False,
        pad_token_id=tokenizer.eos_token_id     # suppresses warning
    )

    answer = outputs[0]["generated_text"].strip()

    return {
        "query": query,
        "answer": answer,
        "context": context,
        "retrieved_docs": retrieved_docs
    }
```

```
# Test generation
test_query = "How do I fine-tune a transformer model?"

retrieved = retrieve_documents(
    query=test_query,
    client=milvus_client,
    collection_name=COLLECTION_NAME,
    embedding_model=embedding_model,
    top_k=3
)

result = generate_answer(
    query=test_query,
    retrieved_docs=retrieved,
    generator=generator
)

print(f"Question: {result['query']}")
print(f"\nAnswer: {result['answer']}")

if result['answer'] and len(result['answer']) > 10:
    print("\n✅ Generation test passed!")
else:
    print("\n❌ Check your generation implementation")
```

```
# Complete RAG pipeline function
def rag_query(
    query: str,
    client: MilvusClient,
    collection_name: str,
    embedding_model: SentenceTransformer,
    generator,
    top_k: int = 5,
    max_new_tokens: int = 256
) -> Dict:
    """
    End-to-end RAG pipeline: retrieve then generate.
    """
    retrieved_docs = retrieve_documents(
        query=query,
        client=client,
        collection_name=collection_name,
        embedding_model=embedding_model,
        top_k=top_k
    )

    result = generate_answer(
```

```
        query=query,
        retrieved_docs=retrieved_docs,
        generator=generator,
        max_new_tokens=max_new_tokens
    )

    return result
```

```
# Test complete pipeline with multiple queries
test_queries = [
    "What is the Trainer class in transformers?",
    "How do I load a dataset from HuggingFace?",
    "What is Gradio used for?"
]

for query in test_queries:
    print(f"\n{'='*60}")
    result = rag_query(
        query=query,
        client=milvus_client,
        collection_name=COLLECTION_NAME,
        embedding_model=embedding_model,
        generator=generator,
        top_k=3
    )
    print(f"Q: {result['query']}")
    print(f"A: {result['answer']}")
```

8. Evaluation with Opik

Automated evaluation of Answer Relevance and Hallucination using **Opik** (Comet ML).

Metrics:

Answer Relevance: Does the generated answer address the query? (0–1, higher is better)
Hallucination: Does the answer contain claims not supported by the context? (0–1, lower is better)

Interpretation:

Relevance > 0.7 → Good retrieval and generation alignment
Hallucination < 0.3 → Effective grounding, prompt guardrails working
If hallucination is high: retrieved context may be off-topic, or model is ignoring the context instructions

```
try:
    import opik
    from opik.evaluation.metrics import AnswerRelevance, Hallucination

    # Configure Opik (reads OPIK_API_KEY from environment)
    opik.configure(use_local=False)

    # Initialize metrics
    relevance_metric = AnswerRelevance()
    hallucination_metric = Hallucination()

    # Evaluation queries
    eval_queries = [
        "How do I use the HuggingFace Trainer class?",
        "What are the different model architectures in transformers?",
        "How do I tokenize text for BERT?",
        "What is the purpose of the attention mask?",
        "How do I save and load a fine-tuned model?"
    ]

    eval_results = []

    for query in eval_queries:
        print(f"Evaluating: {query[:60]}...")

        # Run RAG pipeline
        result = rag_query(
            query=query,
            client=milvus_client,
            collection_name=COLLECTION_NAME,
            embedding_model=embedding_model,
            generator=generator,
            top_k=3
        )

        # Compute metrics
```

```
            relevance_score = relevance_metric.score(
                input=result["query"],
                output=result["answer"]
            )
            hallucination_score = hallucination_metric.score(
                input=result["query"],
                output=result["answer"],
                context=result["context"]
            )

            eval_results.append({
                "query": query,
                "answer": result["answer"],
                "relevance": relevance_score.value,
                "hallucination": hallucination_score.value
            })

        # Summary
        print("\n" + "="*60)
        print("EVALUATION SUMMARY")
        print("="*60)
        avg_relevance = sum(r["relevance"] for r in eval_results) / len(eval_results)
        avg_hallucination = sum(r["hallucination"] for r in eval_results) / len(eval_results)
        print(f"Average Answer Relevance:  {avg_relevance:.3f} (higher is better, target > 0.7)")
        print(f"Average Hallucination:     {avg_hallucination:.3f} (lower is better, target < 0.3)")

        print("\nPer-query results:")
        for r in eval_results:
            print(f"  Q: {r['query'][:50]}...")
            print(f"     Relevance={r['relevance']:.3f} | Hallucination={r['hallucination']:.3f}")

    except ImportError:
        print("Opik not installed or API key not set. Skipping evaluation.")
        print("To enable: pip install opik and set OPIK_API_KEY environment variable.")
        print("Sign up at: https://www.comet.com/")
    except Exception as e:
        print(f"Evaluation error: {e}")
        print("Ensure your OPIK_API_KEY is correct and you have network access.")
```

9. Design Trade-offs & Analysis

Model Selection

Component	Choice	Rationale
Embedding	BAAI/bge-small-en-v1.5	Strong MTEB retrieval scores, 384-dim (compact), fast inference
LLM	Qwen2-1.5B-Instruct	Instruction-tuned, fits in <4GB VRAM, good English comprehension
Vector DB	Milvus Lite (local)	Zero infrastructure overhead, full Milvus API, easy persistence

Chunking Strategy

`chunk_size=1000` chars ≈ 150-200 tokens — captures a full concept without exceeding context
`chunk_overlap=200` (20%) — prevents losing context at boundaries, especially for multi-sentence concepts
Character-level splitting is simple and deterministic; sentence-aware splitting could improve precision further

Retrieval Configuration

`top_k=3` for generation (balances context richness vs. prompt length)
`metric_type=IP` with normalized vectors = cosine similarity at no extra cost
Milvus uses IVF-Flat index by default for Lite — suitable for thousands to low millions of vectors

Hallucination Control

Prompt uses `<context>` XML tags to clearly delineate grounding material
Explicit instruction: "Do NOT make up information that is not in the context"
Fallback: "I don't have enough information" triggers when context is insufficient
Low `temperature=0.7` reduces creativity/hallucination vs default 1.0

Known Limitations

Subset of 500 docs means some queries may fall outside the knowledge base
Character chunking may split mid-sentence; semantic/sentence chunking would improve recall
Qwen 1.5B may struggle with complex multi-hop questions — larger models (7B+) would improve quality

```
# Final pipeline demonstration with an out-of-scope query (hallucination guardrail test)
out_of_scope_query = "What is the weather like on Mars?"
```

```
print("=== HALLUCINATION GUARDRAIL TEST ===")
print(f"Query: {out_of_scope_query}")

result = rag_query(
    query=out_of_scope_query,
    client=milvus_client,
    collection_name=COLLECTION_NAME,
    embedding_model=embedding_model,
    generator=generator,
    top_k=3
)

print(f"\nAnswer: {result['answer']}")
print("\n(Expected: model should say it lacks sufficient information)")
```

```
Both `max_new_tokens` (=256) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refe
=== HALLUCINATION GUARDRAIL TEST ===
Query: What is the weather like on Mars?

Answer: The text does not mention anything about the weather on Mars. It only provides a link to an image of an astronaut ri

(Expected: model should say it lacks sufficient information)
```